

Sotto — 設計仕様

Engineer mentor real-time HUD for Android XR

項目	値
作成日	2026-05-23
status	draft (応募 pitch 用 base)
owner	Yuhtalhara (iharayuhta@gmail.com、個人 indie)
目的	Project Aura Catalyst Program (締切 2026-06-30) 応募の根幹 + 採択後の実装 spec
関連	../../../../project-aura/README.md (Program 概要) / ../../google-xr-design.md (ops 設計累積)
後段	採択判明後 (2026-07-15+) に新 repo ~/work/google/sotto/ を bootstrap、本 doc が implementation plan の base になる
前身	even-memory-bridge (G2 用 frozen PJ) の memory + retrieval 知見を engineer 特化 visual HUD に発展

1. 一言定義 と 北極星

技術質問で詰められた瞬間、過去の設計判断 / 議論 / code を Android XR グラスが視野に sotto voce (ささやくように) 出してくれるアプリ。phone を取り出さず、ガジェットでバレず、user 自身が answer する authenticity を保ったまま、engineer の記憶を拡張する。

北極星: 「多 project context switching で記憶が破綻する engineer / 経営者が、技術会話で credibility を失わないための glasses-native memory companion」になる。generic な meeting AI ではなく、engineer 専門の corpus retrieval + visual HUD に absolute focus。

名前の由来: sotto voce (伊・音楽用語、「ささやくように」)。theater で prompter が舞台袖から sotto voce で役者にセリフを耳打ちする image。AI が代わりに答えるのではなく、user の memory を extend する設計思想を 1 word で示す。副次効果: 「録音されてる感」でなく「私だけ見える / 静かに横にいる」安心感を brand 名で先回り。

2. Positioning (なぜこれが新しいか)

競合状況

層	既存 / 競合	状況
Post-meeting summary SaaS	Otter / Fireflies / Granola / tl;dv / Read.ai	real-time HUD 不可、個人 corpus retrieval なし、phone / browser 前提
Meeting AI sales-focused	Gong / Chorus / Avoma	sales conversation 特化、engineer 痛みに最適化されていない
汎用会議 AI (real-time)	Gemini in Google Meet / Copilot in Teams	汎用 summary / chat、個人 corpus retrieval なし、glasses 不可
Mobile 検索 (phone 取り出し)	ChatGPT / Claude / Perplexity app	phone 取り出し動作が商談中に失礼 / バレる、silently は不可
Glasses 既存 product (audio)	EVEN Realities G2 Conversate / Meta Ray-Ban AI	audio-first、visual context (code / 図) 不可、monochrome 制約
Glasses 既存 product (display)	Meta Orion (research only) / Apple Vision Pro	consumer device 不在 / 価格帯違い / engineer use 最適化されていない
Android XR Display+Audio+Mic + 個人 corpus retrieval + engineer 特化	Aura/Display 未発売、DP4 段階、indie 実装見当たらず	空白
+ user 自身が dogfooder (indie engineer の context switching 痛み)	誰も embedded していない	強 moat

Engineer 特化 vertical が moat な理由

- **technical question の retrieval 価値が突出**: 「なぜ X 設計を選んだ?」「あの bug の原因は?」「あの decision の trade-off は?」 — engineer 質問は具体的 / 構造的で、corpus retrieval が generic 会話より精度高く効く
- **個人 corpus が組成豊富**: engineer は design doc / PR comment / decision log / code / chat / commit message を「書き残す」文化、retrieval 対象が他職種より rich

- **dogfood guarantee:** user 自身が中堅 indie engineer (多 client / 多 project context switching に苦しむ)、1 人企業の「過去の自分が言ったこと忘れる」痛みは毎週直接検証可能
- **判定者刺さりやすい:** Google reviewer は engineer 自身、use case の必要性が 30 秒で直感、経験的に「これ私も欲しい」 と思える
- **未確定 form factor との fit:** Display & Audio glasses は engineer の「会議中も装着可能、周囲に違和感を与えない」 production design と整合、immersive 不要

削った 2 案

- **B. Engineer 全工程 AI 補助 (coding 中 / debugging 中も):** glasses 装着で IDE / terminal を視界拡張、AI suggestion overlay。scope +200%、indie 解像度不足、採択後 v3 以降 → 今期は会話中 retrieval に focus
- **C. Team-level shared corpus:** team 内 corpus を共有して全員の Sotto が同じ context を引く。enterprise SaaS path だが個人 indie の MVP scope 外 → v2 以降

3. ユーザ体験 (Data Flow 物語版)

Phase 1: 装着 + 初期化 (会議前 / 1 日 1 回)

1. 朝 / 会議前にグラス装着、個人 corpus との sync が background 起動
2. 直近 24h の design doc / PR / commit / chat を embedding 化 (差分のみ、数秒 ~ 数分)
3. 視野隅に「Sotto ready」indicator 1 秒、消える

Phase 2: Listening (会話 / 会議中、continuous)

1. user 入力 (Aura button tap / voice “Sotto on” / gesture) で listening 開始
2. mic から PCM rolling buffer (ephemeral、直近 60 秒のみ memory 保持、disk 書込なし)
3. ASR が text stream を継続生成、buffer は逐次 overwrite
4. user は「Sotto が listening 中」indicator (視野隅の小さなドット) で状態認知

Phase 3: 質問検知 → retrieval → HUD render (3-5 秒以内、core)

triggering: - ASR text stream を IntentClassifier が継続評価、「user に向けられた技術質問」を検知 - 検知 trigger: question intonation + technical keyword + 2nd person (“you” / “あなた” / 名指し) + 過去 context への参照 (“the X decision” / 「あの X」) - false positive 削減のため confidence threshold 0.7+

retrieval: - 質問意図 (architecture-decision / debugging / code-review / clarification / general) を分類 - 個人 corpus に hybrid RAG (cosine similarity 0.7 weight + BM25 + LLM rerank) - top-3 結果 (関連 design doc 抜粋 / PR comment / decision log / code snippet / architecture diagram)

HUD render (3 zone、Compose Glimmer): - **上**: 過去 context summary list (3 行、each 60 字以内、出典 file path 含む) - **中**: inline code snippet or decision log highlight (シンタックスハイライト、5-10 行) - **下**: architecture diagram thumbnail (該当 design doc 内の図、tap で拡大)

user 体験: 質問 ~ HUD 表示 まで 3-5 秒。user は「あ、そういえば」と思い出して落ち着いて answer。HUD は user 側だけが見える、相手には phone も device も取り出していないように映る。

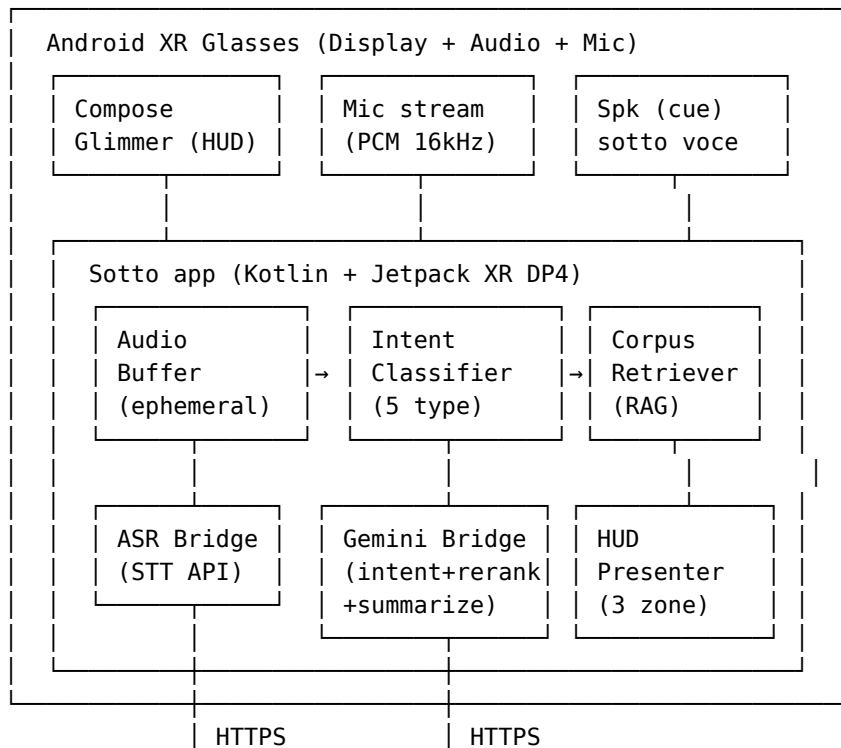
Phase 4: dismiss / 学習 (10-30 秒)

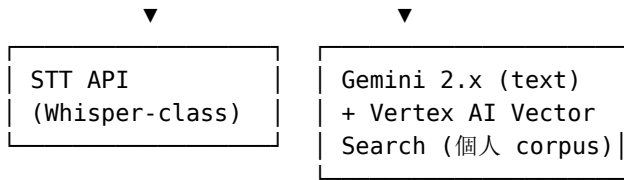
1. user が answer 終了、入力 (gesture / “thanks Sotto” / 30 秒 dwell timeout) で HUD dismiss
2. session log に「質問 → retrieved context → user answered yes/no」を記録 (採択後 v2 で retrieval 精度学習に使用、v1 はログ保存のみ)
3. listening は継続、次質問待ち

Phase 5: 終了 (会議後)

1. user 入力 (Aura button long-press / voice “Sotto off”) で listening 停止
2. ephemeral buffer は全 clear、session log のみ encrypted local storage に保存
3. user は次の会議 / 商談まで待機状態

4. Architecture





6 layer 役割

Layer	内容
Compose Glimmer	視野 HUD (3 zone: summary list / inline code / diagram)
Mic stream	PCM 16kHz ephemeral rolling buffer、disk 書込なし
Spk	sotto voce audio cue (高優先度 retrieval 結果のみ、「思い出した?」程度の短文)
Sotto app	5 module の orchestration
ASR Bridge	STT API client (OpenAI Whisper / Google Cloud Speech-to-Text / on-device option)
Gemini Bridge	intent classification + RAG rerank + summarization の 3 prompt template

核となる原則

- **mic stream は app 内 ephemeral**、ASR API へは chunk 単位送信 (連続 stream 送らない、cost / privacy 対策)
- **個人 corpus は user の Google Cloud project 内 Vector Search**、第三者 access 不可
- **AI は memory extension、user が answer 主体** (HUD は示唆のみ、強制発話なし、dismiss 常時可能)
- **HUD は user only**、mic 録音は ephemeral、「相手から見て透明」を design 軸に

入力 device の表記規約

本 doc 内で「**input tap** / **input long-press** / **input dismiss**」と書く場合は Aura native input device:

- **Aura hardware button** (本体側ボタン)
- **voice command** (mic 経由、「Sotto on」「Sotto off」「thanks Sotto」等)
- **簡易 gesture** (head nod / dwell、SceneCore 経由)

3 つのうちどれが MVP で primary になるかは Aura HW kit 受領後 (2026-08) の検証で確定。design 段階では「input」と generic 表記。商談中 / code review 中の social

acceptability を考慮し、voice command の使用は最小限 (「Sotto on / off」のみ、retrieval 中は非発話操作)。

5. Components (責務境界)

domain layer (Pure Kotlin)

AudioBuffer

- **入力:** PCMFrame stream (16kHz、mono)
- **出力:** AudioChunk (60 秒 rolling、chunk 単位 5 秒)
- **責務:** ephemeral rolling buffer、disk 書込 / 永続化なし、listening 停止で全 clear
- **境界:** memory only、副作用は internal state 更新のみ
- **流用元:** ~/work/even/memory-bridge/apps/even-plugin/src/buffer.ts (改修要、Aura SDK wrapper 再作)

IntentClassifier

- **入力:** TranscriptStream (ASR の text chunk)
- **出力:** Intent (type: architecture-decision / debugging / code-review / clarification / general、confidence 0-1、detected_topic)
- **責務:** 「user に向けられた技術質問」を継続評価、false positive 削減のため confidence ≥ 0.7 のみ retrieval 発火
- **境界:** pure function、副作用なし。LLM call は GeminiBridge に委譲
- **流用元:** ~/work/even/memory-bridge/services/llm.py の question_type enum (6 type → engineer 特化 5 type に再定義)

CorpusRetriever

- **入力:** Intent + topic keyword
- **出力:** RetrievalResult (top-3、each: snippet / file_path / heading_path / score)
- **責務:** 個人 corpus に hybrid RAG (cosine 0.7 + BM25 + LLM rerank)、design doc / PR / decision log / code / chat 横断
- **境界:** corpus 読み取りのみ、書き込みは別 module (CorpusIngester、v2 以降)
- **流用元:** ~/work/even/memory-bridge/services/rag.py (そのまま env 置換、完成品)

presentation layer (Jetpack XR)

HudPresenter

- **入力:** RetrievalResult
- **出力:** Compose Glimmer 描画
- **責務:** 視野 HUD 3 zone レイアウト
 - **上:** summary list 3 行 (each 60 字、出典 file path 含む)
 - **中:** inline code snippet or decision highlight (シンタックスハイライト、5-10 行)

- ・ **下**: architecture diagram thumbnail (tap で拡大)
- ・ **流用元**: ~/work/even/memory-bridge/src/hud.ts 3-page layout (画面 size 調整、Compose Glimmer 書き換え)

AudioCue

- ・ **入力**: 高優先度 RetrievalResult
- ・ **出力**: sotto voce TTS (短文 1-2 単語、「思い出した？」程度)
- ・ **責務**: HUD だけでは気付かない user のため audio nudge、ただし周囲に聞こえないよう極小音量 + 短文
- ・ **境界**: 商談中の social acceptability 優先、多用しない

infrastructure layer

ASRBridge

- ・ **入力**: AudioChunk (5 秒 PCM)
- ・ **出力**: TranscriptChunk (text + timestamp + speaker hint)
- ・ **責務**: STT API call (OpenAI Whisper-class、v1 は cloud、v2 で on-device 検討)、chunk 単位送信
- ・ **流用元**: ~/work/even/memory-bridge/services/stt.py (そのまま env 置換)

GeminiBridge

- ・ **prompt template 3 種**:
 - ▶ classifyIntent (TranscriptChunk → Intent、engineer-specific schema)
 - ▶ rerankRetrieval (Intent + raw RAG results → top-3 ranked)
 - ▶ summarizeForHud (RetrievalResult → 60 字 summary list × 3)
- ・ **retry / rate-limit / cost meter** (1 月 5000 円上限 alert)
- ・ **責務**: API 呼び出しを 1 箇所に集約、cost monitor / API key 管理 / retry 一元化

MaskingLayer

- ・ **入力**: TranscriptChunk (生 text)
- ・ **出力**: MaskedTranscript (PII / 機密 / 客先固有 identifier を placeholder 置換、audit log 記録)
- ・ **責務**: LLM / STT API への送信前 mask、user の privacy + 客先 NDA 保護
- ・ **流用元**: ~/work/even/memory-bridge/ masking layer (YAML frontmatter ベース、そのまま流用)

7 個 (domain 3 + presentation 2 + infra 3 → 8 components) に絞った理由

- ・ AudioBuffer / IntentClassifier / CorpusRetriever の 3 way 分割は責務軸 (入力 / 意図 / 知識) と一致、後で個別差し替え可
- ・ HUD と Audio を分離: Display & Audio glasses は別出力 channel、並列 dev 可
- ・ ASRBridge / GeminiBridge / MaskingLayer を infra layer に集約: 外部 API + privacy 境界の管理を一元

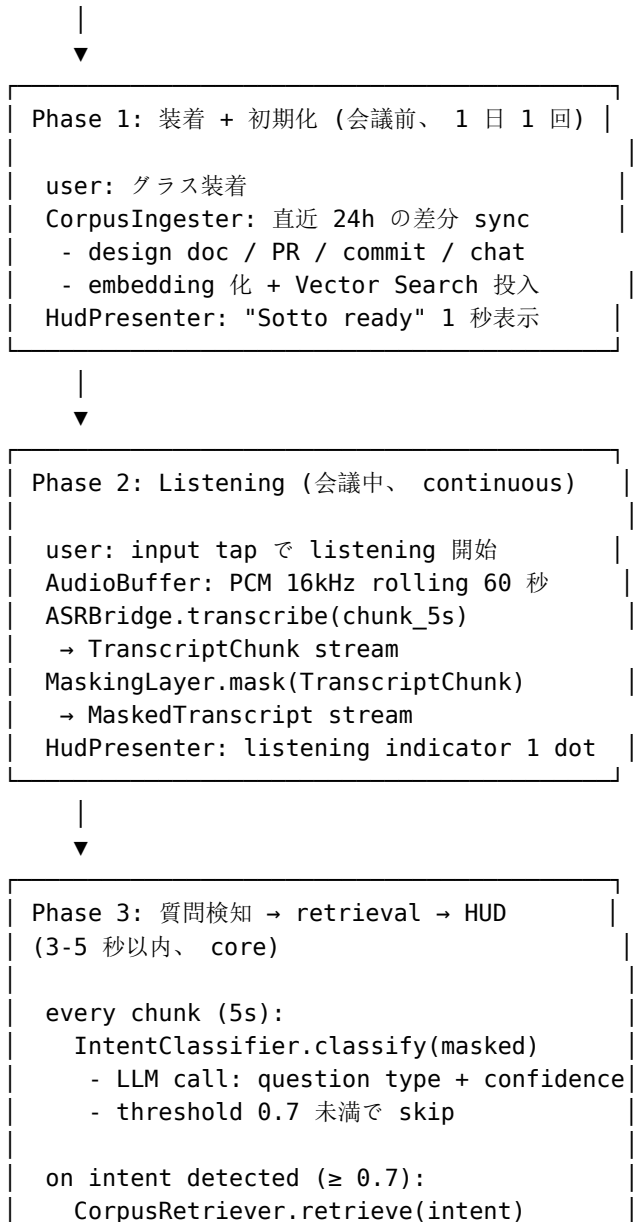
- MaskingLayer を独立 component に: privacy が design 軸の中核、enforced 経路を明示

Engineer 特化部分の閉じ込め

vertical 依存は **IntentClassifier** の **type enum** と **CorpusRetriever** の対象 **source schema** の 2 箇所のみ。他は generic、将来「経営者 / 営業 / 医療」等別 vertical 追加時ここだけ差し替え。

6. Data Flow (詳細)

[T0] user 装着、listening 待機



```

1. hybrid RAG (cosine 0.7 + BM25)
2. raw top-10 candidates
GeminiBridge.rerankRetrieval(...)
→ top-3 RetrievalResult
GeminiBridge.summarizeForHud(...)
→ summary list × 3 + code highlight
+ diagram thumbnail
HudPresenter.show(RetrievalResult)
- 上: summary list 3 行
- 中: inline code / decision
- 下: architecture diagram
AudioCue.play(sotto voce) if HIGH

```

|



```

Phase 4: dismiss / 学習 (10-30 秒)

user: answer 中、 HUD 視認
user: input gesture or 30s dwell
→ HudPresenter.dismiss()
SessionLog.record(intent, retrieved,
                  user_answered_yn)
listening は継続、 next 質問待ち

```

|



```

Phase 5: 終了 (会議後)

user: input long-press で listening 停止
AudioBuffer.clear() (全 PCM erase)
SessionLog: encrypted local storage 保存
HudPresenter: "Sotto off" 1 秒、 消える

```

重要設計判断

- **STT + Gemini call は chunk 単位**、prompt template 化、cost 予測しやすく。1 会議 60 分 = 平均 60-120 calls (chunk 5 秒 × 12 / min × 60 min 内で intent 検知時のみ retrieval)
- **mic 録音は ephemeral**、disk 書込なし、listening 停止で全 clear (privacy core principle)
- **MaskingLayer 必須経路**: ASR 後 / LLM 送信前で必ず通過、audit log で「何が mask されたか」 user が後から確認可
- **HUD は user only**: glasses の display は user 側 only に映る、相手から見えない (Display & Audio glasses HW 前提)

- **HUD dismiss は 3 方法常時可能:** gesture / voice / dwell timeout、 「読みたくない」を強制しない
-

7. Error / Edge cases

分類	ケース	対処
入力	mic 入力 が 騒音 環境 (cafe / 工事音) で ASR 精度低下	ASR confidence threshold 0.6+、未満は HUD 出さず skip。user に「環境ノイズ大」amber 1 秒表示
	複数話者の overlap	speaker diarization (v1 は best-effort、v2 で voice ID)、user 向け質問の判定は 2nd person + intonation で fallback
	質問でない発話 (“X is great” 等)	IntentClassifier の question intonation feature で除外、false positive < 10% target
意図抽出	engineer 質問 でなく 雑談 (天気 / 食事)	type “general” に分類、confidence ≥ 0.7 でも HUD 出さない (engineer 特化 vertical の絞り)
	過去 context への参照なし (一般技術質問、例「Kotlin の coroutine って何?」)	個人 corpus 不要、「Sotto は personal context 専用」として HUD 出さず skip、user は phone or 自力で answer
retrieval	corpus に該当 context なし (initial setup 1 週間以内 / 新規 project)	top score < 0.5 で HUD 「該当 context なし」1 秒表示、user は素直に「思い出せない」と answer
	retrieval 結果が古すぎる (1 年前の design doc)	freshness penalty (時間経過で score 減衰)、user が「過去 / 現在」不一致を見抜けるよう 出典日付を summary に明示
API	STT / Gemini API 落ちる / network 切れる	HUD 「Sotto unavailable」amber、listening は継続 (ephemeral buffer 保持)、復旧で再開
安全	rate limit / cost 上限到達 (Gemini の cost 超過) / corpus 出る古 (false positive) 失敗 (false negative)	cost meter で月 5000 円 上限 (Gemini の cost 超過) / corpus 出る古 (false positive) 失敗 (false negative) 発生時は Find My Device

設計原則

- **AI は memory extension、user が answer 主体**。HUD は示唆のみ、強制発話なし、dismiss 常時可能
 - **privacy / 社会受容 を design core に**: ephemeral / masking / user-only display / 1 操作で silent / encrypted log
 - **degraded mode を多段** (full → API skip → audio only → silent monitoring → off)、完全停止しない
 - **cost meter は app 内に常時表示** (画面下に「今月: 1,240 円 / 5,000 円」)、indie が課金事故防止
-

8. Testing strategy

	Layer	何を / どう	目標
	Unit (Kotlin)	AudioBuffer rolling / IntentClassifier confidence threshold / MaskingLayer placeholder 置換 / cost meter / input event handler	JUnit5 + MockK、80% line cov
		IntentClassifier: 5 type 分類精度 (architecture-decision / debugging / code-review / clarification / general)、各 type 50+ test cases	type 別 precision ≥ 0.8
		HudPresenter 3 zone レイアウト境界 / overflow handling	Compose preview + screenshot diff
	Integration	ASRBridge: chunk 単位 STT API call を recorded fixture で再生	VCR pattern、API key 不要、CI で回せる
		GeminiBridge: 3 prompt template (classifyIntent / rerankRetrieval / summarizeForHud) を fixture 再生	全 template の response schema 検証
		CorpusRetriever: 個人 corpus (mock) 100 docs に対する hybrid RAG 精度、top-3 recall ≥ 0.7	自前 fixture、repo 内 tests/fixtures/corpus/
	Emulator	エラーパス: rate-limit / timeout / 不正 JSON / 部分応答 を mock で再現	全 error case の retry / fallback 検証
		HUD layout (Compose Glimmer): 3 zone が視野内収まる / dark mode / 視角境界	Studio preview + emulator screenshot diff
		mic mock (録音 WAV stream)、質問検知 → retrieval pipeline 通る	各 phase 通過確認
実績 (Automated testing 後)	Android Studio 15 (受領後)	開発環境の自動化と品質向上のための取り組み - CI/CDパイプラインの構築 - テストケースの自動化 - ユーザー体験の改善 UX 計測	開発効率の向上と品質向上 - テスト結果の可視化 - エラーの迅速な検出 - フォールバックの自動的な処理 - ユーザーフィードバックの活用 negative 検出

MVP の品質ゲート (Play 公開 2027-04-30 の条件)

1. user 自身 + 友人 engineer 2 名で **dogfood 50 回以上** + crash-free
2. retrieval **precision ≥ 0.8 / recall ≥ 0.7** (主要 type architecture-decision / debugging、user 評価)
3. battery: **90 分 listening で残 $\geq 40\%$**
4. cost: **1 会議 (60 分) あたり STT + Gemini ≤ 100 円**
5. onboarding: 「初回起動 → 1 つ目の有用 retrieval」を **1 時間未満** で達成
6. privacy: **MaskingLayer 経由率 100% (LLM 送信前)**、audit log で user が事後検証可

意図的に削ったもの

- 自動 UI test (Espresso / UI Automator) は HUD 依存で fragile、indie scope 外
 - 多言語 (日 + 英 のみ、他は v2 以降)
 - 負荷試験 (single-user app、不要)
 - enterprise SSO / team corpus 共有 (v2 以降)
 - on-device ASR (v1 は cloud、v2 以降検討)
-

9. 応募 narrative (Catalyst form 落とし込み)

App 名 (working title)

Sotto — Engineer mentor real-time HUD for Android XR

- Sotto = sotto voce (伊・音楽用語、「ささやくように」)、「AI が静かに横で memory extension」暗喩
- 商標 check 済 (2026-05-23): AI/dev tool 領域に衝突なし、採用
- 副次効果: 「録音されてる感」でなく「私だけ見える / 静かに横にいる」安心感を brand 名で先回り
- 候補別案 (商標衝突で却下): Prompter (prompter.ai 等被り) / Kuroko (Kuroco RAG / GitHub Kuroko stealth AI assistant 被り)

Vertical Alignment (Category multi-select)

応募 form 実物 (../project-aura/application/form-structure-2026.md Page 3 Category) の選択肢に合わせて 3 つ multi-select:

- ☒ **Productivity** (主軸、engineer の会話中 memory extension = 生産性 tool)
- ☒ **Enterprise** (engineer / 経営者 / 商談 = enterprise context)
- ☒ **Education** (junior engineer の standup / code review 用語追従 = 学習支援)

“A glasses-native memory companion for engineers and tech founders, retrieving past technical context (design docs, PR comments, decision logs) in real time during conversations — a category currently unaddressed on any XR platform.”

Use Case (one paragraph)

Engineers and tech founders carry too much technical context to recall on demand. Mid-conversation, when asked “why did you choose X over Y?” or “what was the trade-off in that architecture?”, they fumble — not because they don’t know, but because the past decision is buried in design docs, PR comments, and decision logs they wrote months ago. Phone lookup is socially impossible in a serious meeting; AI on a phone signals weakness. Sotto captures the question via the glasses’ microphone, classifies it as a technical retrieval request, runs hybrid RAG over the user’s personal corpus, and renders the relevant past context — code snippets, decision logs, architecture diagrams — directly in the user’s field of view. The user answers in their own voice, with their own authority. The phone stays in the pocket. The other party sees no device, no glance away, no hesitation.

Supporting files (form Page 3 upload, max 5 / 100 MB each)

応募 form の attachment 欄を「indie 1 人が design 完了済」を示す winning move として活用:

1. **2026-05-23-sotto-design.md** → **PDF** (本 spec、11 section / 約 600 行)
2. **Even repo 流用 component リスト (2026-05-23-sotto-even-component-reuse.md)** → **PDF** (file-level inventory、流用率 60-70% の証拠)
3. (optional) architecture diagram を画像 export (本 spec § 4 から)

Developer Readiness

質問 (form Page 1+2 対応)	回答
Company Name (form Page 1)	Yuhta Ihara (個人 indie)
Job title (form Page 1)	Indie Developer
Country (form Page 1)	Japan
Region (form Page 1)	APAC
Team size (form Page 2 自由記述)	Solo indie developer (YuhtaIhara、個人)
Prior XR experience	EVEN Realities G2 daily user since 2026-03 (約 2-3 ヶ月)、主に Conversate (会話 augmentation)、HUD UX を消費者として深く理解
Prior Android experience	Kotlin / Android 触れて 1 年未満、採択後 9 ヶ月 window で習得 + Sotto MVP 構築
Domain expertise	中堅 indie engineer 自身、多 client / 多 project context switching の痛みを daily で体感 (dogfood guarantee)
Open source / portfolio (form Page 2 Existing apps)	https://github.com/Yuhtalhara (pinned: even-memory-bridge + google-xr-ops、profile README から sharepoint-rag-azure + sharepoint-rag-lite を link)
Concept validation	Engineer memory extension は user 自身の daily 痛み、前身 PJ even-memory-bridge (G2 用 frozen) で 5 ヶ月の dogfood 検証済、Aura で visual HUD として完成形に到達
Development progress (form Page 3 0-10 scale)	2 (design + spec 完了 + 前身 PJ memory-bridge で RAG/STT pipeline 完成品所持 + PoC mock HUD 完了 / DP4 emulator の Compose Glimmer 知見)

Funding Requirements

Request: No grant required (応募 form Page 4 Grant amount で選択)

理由:

- Sotto の MVP cost (STT + Gemini API + Vector Search) は indie 1 人の試算で 年 **10-25 万円** 程度、\$10K (≈ 150 万円) は overshooting

- “Limited grants at each level exist” 文言、 控えめ姿勢 (No grant) は採用率を上げる方向
- HW kit (Display & Audio glasses prototype) + technical resources へのアクセスのみで MVP 9 ヶ月完成可能
- 前身 PJ memory-bridge (TypeScript / Python 実装、 frozen) を Kotlin 移植 + Aura SDK 統合する開発で、 ゼロから書かない (Even repo 流用率 60-70%)
- 採択後に grant 必要が出てくれば、 program 内 milestone-based 追加申請の道があるか別途確認

“We are not requesting grant funding for this application. The MVP scope (engineer-specific question retrieval HUD with hybrid RAG) reuses 60-70% of our prior project (even-memory-bridge, frozen but battle-tested) and can be self-funded by a solo developer. We’re applying for hardware kit access and technical guidance — these are the multiplicative resources we cannot replicate independently.”

Form Factor Selection (排他、 form Page 3)

応募 form 実物は **Wired XR glasses / Display & Audio glasses の排他 2 択**。

選択: ☒ **Display & Audio glasses** (production design prototype)

根拠: - Sotto core (Display HUD + Audio cue + Mic) は production form factor (audio glasses + display glasses) と直接 fit - Wired XR glasses (Aura) は immersive 有線 device、 商談 / code review / 1:1 で wired は実用不可 (社会的にも実用的にも) - Display & Audio path を選ぶことで MVP がそのまま production 端末に乗る (Warby Parker Intelligent Eyewear 2026 秋発売 / 後続 production design と form factor 同じ) - 「商談中に装着していても違和感ない」 社会受容性が core requirement、 immersive XR は selection 不能

Tech Stack (form Page 2 multi-select)

- ☒ **Android with Compose** (= Kotlin + Jetpack XR SDK DP4 + Jetpack Compose Glimmer)

詳細:

- **Primary:** Kotlin + Jetpack XR SDK (DP4 → Beta)
- **UI:** Jetpack Compose Glimmer
- **Audio:** Android MediaRecorder (PCM 16kHz ephemeral)
- **ASR:** OpenAI Whisper API (cloud、 v2 で on-device 検討)
- **LLM:** Gemini 2.x text via Vertex AI (intent classification + rerank + summarization)
- **Vector Search:** Vertex AI Vector Search (個人 corpus、 user の Google Cloud project)
- **Auth:** Google OAuth 2.0 (個人 Google Cloud project)
- **Distribution:** Google Play (Android XR target)

Unity-specific 項目 (Unity version / render pipeline / Graphics API) は **N/A** (Compose 経由のため)

Target Launch Regions (form Page 3 multi-select)

- ☒ Global

Engineer / tech founder の memory extension は世界中で需要、言語対応は MVP で日 + 英、他は v2 以降。JAPAC 限定にしない。

Timeline (6-12 ヶ月 Play 公開逆算)

2026-07-15 採択判明
2026-08 HW kit 受領、sotto repo bootstrap
2026-08-09 DP4 で UI prototype (HUD 3 zone layout、emulator)
2026-09-10 Audio + ASR pipeline (Even repo memory-bridge から Kotlin 移植)
2026-10-11 Intent classification + Retrieval pipeline (Even repo RAG 移植 + engineer 特化 type)
2026-12 実機 alpha (Phase 1-3 完成、自己 dogfood 開始)
2027-01-02 Privacy / Masking / Session log 統合
2027-03 beta、友人 engineer dogfood 2 名
2027-04-30 Play 公開 (MVP: engineer 質問 5 type 対応、個人 corpus setup wizard)

10. 将来拡張 / 採択後 follow-up

v2 以降の roadmap (pitch では 1-2 行触れる程度)

- **Team corpus 共有**: enterprise SaaS path、team 内で corpus を共有して全員の Sotto が同じ context 引く (削った案 C)
- **Engineer 全工程 AI 補助**: 会話中だけでなく coding 中 / debugging 中も glasses overlay (削った案 B、scope 過大)
- **Multi-vertical**: 経営者 / 営業 / 医療 / 法律 等の persona に対応 (IntentClassifier + CorpusRetriever schema 差し替え)
- **On-device ASR / LLM**: privacy 強化、offline 動作、cost 削減
- **Voice ID + diarization**: 複数話者の質問判定精度向上

採択後 follow-up (今 session では作業しない)

1. repo 分離 (2026-07-15+ 採択判明後)

採択判明と同時に新 repo ~/work/google/sotto/ を個人 workflow template から bootstrap:

```
~/work/google/sotto/ ← 新 repo
├ app/                  ← Kotlin Android app
├ docs/
└ README.md
```

google-xr-ops/project-aura/apps/ placeholder は削除 or “see sotto repo” 1 行に置換。
ops 側は pitch / design doc / 振り返り保管に専念。

2. Even 関連の整理 (2026-07-15+ 採択判明後)

採択判明後に「even-memory-bridge との関係明確化」 task を起票:

- even-memory-bridge は frozen reference として保持 (portfolio inclusion 済)
- Sotto は「Aura visual HUD + engineer 特化」 として axis 違い positioning
- Even Realities 公式 (Even Vendor) との競合に発展しないよう、「G2 = audio HUD = ambient 軽負荷」 / 「Sotto = visual HUD = 商談 / code review 集中 use」 を明示
- 採択 credibility を活かして Even に協業提案する path も視野 (camera 無し版 = audio only Sotto を G2 + R1 ring で実装) は v3 以降検討

戦略的順序: Aura 採択 (Google credibility) → Even との対話。 逆順は反応時間読めず Aura 締切を脅かす risk。

3. Catalyst pitch では「multi-platform vision」 1 行のみ言及

- pitch 本文 (Section 9 既載) に “future expansion to audio-glass and team-shared variants” として軽く触れる
- focus 散漫印象を避けるため、 1st version Android XR + engineer solo フォーカスは死守

11. 関連 docs

- Catalyst Program 概要: ../../../../project-aura/README.md
- ops 設計累積: ../../google-xr-design.md
- 公式ランディング: <https://developer.android.com/develop/xr/catalyst> (短縮 g.co/dev/catalyst)
- 応募フォーム: https://docs.google.com/forms/d/e/1FAIpQLSesZIBGg010S0K9Xm5ZvxTt2WMzHGnKtBUYEHBrzd0_uwmHEA/viewform
- 応募締切: **2026-06-30 23:59 PDT** (= JST 2026-07-01 15:59)
- 結果通知: 2026-07-15 まで
- Even repo 流用元 (user 自身の indie 個人著作物、倫理 OK): ~/work/even/conversate-ops + ~/work/even/memory-bridge